

Sprint 3 Implementation Plan: Complex Mechanics Rings (Tier 3)

SPRINT 3 OVERVIEW

Goal: Implement 6 complex rings requiring new subsystems for spell storage, counterspells, spell slot manipulation, and regeneration mechanics.

Duration: 3-4 weeks (22 working days)

Rings:

- Ring of Counterspells
- Ring of Spell Storing (Minor)
- Ring of Spell Storing (Major)
- Ring of Wizardry (I-IV)
- Ring of Regeneration

Complexity Level: High - requires new core systems that extend existing spell and character mechanics

RINGS TO IMPLEMENT

1. Ring of Counterspells (4,000 gp)

Mechanics:

- Store one spell (maximum 6th level)
- Automatically triggers to counter that specific spell when cast at wearer
- Functions as Dispel Magic against that spell only
- One-time use per stored spell

Complexity: ★★☆☆

2. Ring of Spell Storing, Minor (18,000 gp)

Mechanics:

- Store up to 3 spell levels
- Anyone can cast stored spells (even non-casters)
- Uses stored caster level and DC
- Examples: one 3rd-level spell, or three 1st-level spells

Complexity: ★★★★★

3. Ring of Spell Storing, Major (200,000 gp)

Mechanics:

- Store up to 5 spell levels
- Same mechanics as Minor version
- Examples: one 5th-level spell, or five 1st-level spells

Complexity: ★★★★★

4. Ring of Wizardry I-IV

Type I (20,000 gp): Double 1st-level spell slots

Type II (40,000 gp): Double 2nd-level spell slots

Type III (70,000 gp): Double 3rd-level spell slots

Type IV (100,000 gp): Double 4th-level spell slots

Mechanics:

- Only affects arcane casters (Wizard, Sorcerer, Bard)
- Doubles prepared slots (Wizard) or daily slots (Sorcerer/Bard)
- Does NOT grant new spells known
- Stacks with bonus slots from high ability scores

Complexity: ★★★★★

5. Ring of Regeneration (90,000 gp)

Mechanics:

- Heal 1 HP per character level per hour
- Restore severed limbs in 3d6 rounds
- Cannot die from HP damage (remains stable at negative HP)
- Still vulnerable to massive damage, death effects, Constitution damage

Complexity: ★★★★★

NEW SYSTEMS REQUIRED

SYSTEM 1: Spell Storage System

Purpose: Store spells in items with metadata, allowing later casting with original parameters

Components:

```
public class StoredSpell
{
    public string SpellName;
    public int CasterLevel;
    public int SaveDC;
    public int SpellLevel;
    public int SpellID; // For unique identification
}

// Add to ItemData.cs
public List<StoredSpell> StoredSpells;
public int MaxStoredSpellLevels; // 3 for Minor, 5 for Major, 1 for Counterspells
public bool IsCounterspellRing; // Special flag for counterspell behavior
```

Features:

- Store spell (cast into ring, consuming spell slot)
- View stored spells with metadata
- Cast stored spell (anyone can use)
- Remove stored spell

- Capacity tracking (total spell levels)
- Validation (max spell level, capacity limits)

Implementation Details:

```
public class SpellStorageManager : MonoBehaviour
{
    public bool StoreSpell(ItemData ring, Spell spell, int casterLevel, int saveDC)
    {
        if (ring.StoredSpells == null)
            ring.StoredSpells = new List<StoredSpell>();

        // Check capacity
        int currentLevels = ring.StoredSpells.Sum(s => s.SpellLevel);
        if (currentLevels + spell.Level > ring.MaxStoredSpellLevels)
        {
            Debug.Log("Ring is full!");
            return false;
        }

        // Store spell
        StoredSpell stored = new StoredSpell
        {
            SpellName = spell.Name,
            CasterLevel = casterLevel,
            SaveDC = saveDC,
            SpellLevel = spell.Level,
            SpellID = UnityEngine.Random.Range(1000, 9999)
        };

        ring.StoredSpells.Add(stored);
        return true;
    }

    public bool CastStoredSpell(ItemData ring, int spellID, Character caster, Character target)
    {
        StoredSpell stored = ring.StoredSpells.FirstOrDefault(s => s.SpellID == spellID);
        if (stored == null) return false;

        // Cast spell using stored CL and DC
        SpellManager.Instance.CastStoredSpell(stored, caster, target);

        // Remove spell from storage
        ring.StoredSpells.Remove(stored);
        return true;
    }
}
```

Time Estimate: 4-5 days

SYSTEM 2: Counterspell Trigger System

Purpose: Automatically trigger counterspell when specific spell is cast at wearer

Components:

```
// Add to ItemData.cs
public string CounterspellTrigger; // Spell name to counter

// In SpellManager.cs - modify CastSpell()
public bool CastSpell(Spell spell, Character caster, Character target)
{
    // Before spell resolution, check for counterspell
    if (CheckCounterspell(spell, target))
    {
        Debug.Log($"{spell.Name} was countered by Ring of Counterspells!");
        return false; // Spell fizzles
    }

    // Normal spell resolution...
}

private bool CheckCounterspell(Spell spell, Character target)
{
    foreach (ItemData ring in target.EquippedRings)
    {
        if (ring.StoredSpells != null && ring.IsCounterspellRing)
        {
            StoredSpell counterspell = ring.StoredSpells.FirstOrDefault(
                s => s.SpellName == spell.Name
            );

            if (counterspell != null)
            {
                // Counter successful - remove stored spell
                ring.StoredSpells.Remove(counterspell);

                // Visual feedback
                EffectManager.Instance.PlayCounterspellEffect(target);

                return true;
            }
        }
    }
    return false;
}
```

Features:

- Store counterspell in ring (UI to select spell)
- Automatic triggering on spell cast
- One-time use per stored spell
- Visual/audio feedback
- Integration with spell casting pipeline

Time Estimate: 2-3 days

SYSTEM 3: Spell Slot Doubling

Purpose: Double spell slots for specific level based on Ring of Wizardry type

Components:

```

// Add to CharacterStats.cs
public int GetBonusSpellSlots(int spellLevel)
{
    // Check equipped rings for Ring of Wizardry
    foreach (ItemData ring in EquippedItems.Where(i => i.IsRing))
    {
        if (ring.ItemName.Contains("Ring of Wizardry"))
        {
            int ringLevel = GetWizardryRingLevel(ring.ItemName);
            if (ringLevel == spellLevel)
            {
                // Double the base slots for this level
                int baseSlots = GetBaseSpellSlots(spellLevel);
                return baseSlots; // Return base as bonus (doubles total)
            }
        }
    }
    return 0;
}

private int GetWizardryRingLevel(string ringName)
{
    if (ringName.Contains("I") && !ringName.Contains("II")) return 1;
    if (ringName.Contains("II") && !ringName.Contains("III")) return 2;
    if (ringName.Contains("III") && !ringName.Contains("IV")) return 3;
    if (ringName.Contains("IV")) return 4;
    return 0;
}

// Modify spell slot calculation
public int GetSpellSlotsForLevel(int level)
{
    // Only arcane casters benefit
    if (!IsArcaneCaster()) return 0;

    int baseSlots = CalculateBaseSlots(level);
    int bonusFromAbility = GetBonusSlotsFromAbility(level);
    int bonusFromRing = GetBonusSpellSlots(level);

    return baseSlots + bonusFromAbility + bonusFromRing;
}

public bool IsArcaneCaster()
{
    return CharacterClass == "Wizard" ||
           CharacterClass == "Sorcerer" ||
           CharacterClass == "Bard";
}

```

Features:

- Detect equipped Ring of Wizardry
- Identify which level to double
- Only affect arcane casters
- Stack with ability score bonuses
- Update spell preparation UI
- Update spell casting UI

UI Integration:

```
// In SpellBookUI.cs
public void UpdateSpellSlotDisplay()
{
    for (int level = 1; level <= 9; level++)
    {
        int total = character.GetSpellSlotsForLevel(level);
        int used = character.GetUsedSpellSlots(level);
        int bonus = character.GetBonusSpellSlots(level);

        if (bonus > 0)
        {
            slotText[level].text = $"{used}/{total} (+{bonus} from Ring of Wizardry)";
            slotText[level].color = Color.cyan; // Highlight doubled slots
        }
        else
        {
            slotText[level].text = $"{used}/{total}";
        }
    }
}
```

Time Estimate: 3-4 days

SYSTEM 4: Regeneration System

Purpose: Continuous healing over time with special death prevention mechanics

Components:

```

public class RegenerationEffect : MonoBehaviour
{
    private Character _character;
    private float _hourTimer = 3600f; // 1 hour in seconds
    private bool _isActive = false;

    public void Initialize(Character character)
    {
        _character = character;
        _isActive = true;
    }

    void Update()
    {
        if (!_isActive || _character == null) return;

        _hourTimer -= Time.deltaTime;

        if (_hourTimer <= 0)
        {
            // Heal 1 HP per character level
            int healAmount = _character.Level;
            _character.Heal(healAmount);

            Debug.Log($"{_character.Name} regenerated {healAmount} HP");

            // Reset timer
            _hourTimer = 3600f;
        }
    }

    public void OnDisable()
    {
        _isActive = false;
    }
}

// Add to Character.cs
public bool HasRegenerationRing()
{
    return EquippedItems.Any(i => i.ItemName.Contains("Ring of Regeneration"));
}

public bool CanDieFromHPLoss()
{
    // Cannot die from HP damage if wearing Ring of Regeneration
    return !HasRegenerationRing();
}

// Modify TakeDamage() in Character.cs
public void TakeDamage(int damage, DamageType type = DamageType.Normal)
{
    CurrentHP -= damage;

    if (CurrentHP <= 0)
    {
        if (CanDieFromHPLoss())
        {
            // Normal death
            Die();
        }
        else
    }
}

```

```

        {
            // Stabilize at negative HP
            Debug.Log($"{Name} is unconscious but stable (Ring of Regeneration)");
            SetUnconscious();
        }
    }

    // Still check for massive damage
    if (damage >= 50)
    {
        MassiveDamageCheck();
    }
}

// Limb restoration
public IEnumerator RestoreSeveredLimb()
{
    int rounds = Random.Range(3, 19); // 3d6 rounds
    float roundTime = 6f; // 6 seconds per round

    yield return new WaitForSeconds(rounds * roundTime);

    Debug.Log($"{Name}'s severed limb has been restored!");
    // Restore limb functionality
}

```

Features:

- Hourly healing (1 HP per level per hour)
- Real-time or accelerated time tracking
- Severed limb restoration (3d6 rounds)
- Cannot die from HP damage
- Still vulnerable to:
 - Massive damage (50+ in one hit)
 - Death effects (Finger of Death, etc.)
 - Constitution damage to 0
- Visual indicator of regeneration status

Performance Optimization:

```
// Alternative: Use coroutine instead of Update()
public class RegenerationEffect : MonoBehaviour
{
    private Character _character;

    public void Initialize(Character character)
    {
        _character = character;
        StartCoroutine(RegenerationCycle());
    }

    private IEnumerator RegenerationCycle()
    {
        while (true)
        {
            yield return new WaitForSeconds(3600f); // 1 hour

            if (_character != null && _character.HasRegenerationRing())
            {
                int healAmount = _character.Level;
                _character.Heal(healAmount);
            }
        }
    }
}
```

Time Estimate: 3-4 days

IMPLEMENTATION PHASES

PHASE 1: Counterspell Ring (Days 1-3)

Objectives:

- Build counterspell trigger system
- Implement Ring of Counterspells data
- Create UI to store counterspell
- Test auto-counter mechanics
- Integration with spell casting pipeline

Tasks:

- [] Add `IsCounterspellRing` flag to `ItemData`
- [] Modify `SpellManager.CastSpell()` to check counterspells
- [] Create UI for storing counterspell in ring
- [] Add visual/audio feedback for countering
- [] Test with various spell levels (1st-6th)
- [] Verify one-time use consumption

Deliverables:

- `CounterspellManager.cs`
 - Ring of Counterspells in `RingFactory`
 - Counterspell storage UI panel
-

PHASE 2: Spell Storage Foundation (Days 4-8)

Objectives:

- Build spell storage data structure
- Create storage UI (view/manage spells)
- Implement “cast into ring” mechanic
- Test capacity tracking
- Verify metadata preservation

Tasks:

- [] Create StoredSpell class
- [] Add spell storage fields to ItemData
- [] Implement SpellStorageManager
- [] Create UI panel to view stored spells
- [] Add “Cast into Ring” button to spell UI
- [] Implement spell casting from storage
- [] Test capacity limits (3 levels for Minor, 5 for Major)
- [] Verify CL and DC preservation
- [] Add validation for max spell level

Deliverables:

- SpellStorageSystem.cs
 - StoredSpell.cs
 - Spell storage UI panel
 - Updated SpellManager integration
-

PHASE 3: Spell Storing Rings (Days 9-11)

Objectives:

- Implement Ring of Spell Storing (Minor)
- Implement Ring of Spell Storing (Major)
- Test casting stored spells
- Verify anyone can use stored spells

Tasks:

- [] Create Ring of Spell Storing (Minor) in RingFactory
- [] Create Ring of Spell Storing (Major) in RingFactory
- [] Test storing multiple spells
- [] Test casting with non-caster character
- [] Verify original CL/DC are used
- [] Test capacity management
- [] Test spell removal
- [] Save/load testing for stored spells

Deliverables:

- Ring of Spell Storing (Minor) item
 - Ring of Spell Storing (Major) item
 - Test results documentation
-

PHASE 4: Spell Slot Doubling (Days 12-16)

Objectives:

- Modify spell slot calculation system
- Implement Ring of Wizardry I-IV
- Test with Wizard/Sorcerer/Bard
- Update spell UI

Tasks:

- [] Add GetBonusSpellSlots() to CharacterStats
- [] Add IsArcaneCaster() check
- [] Modify GetSpellSlotsForLevel() calculation
- [] Implement Ring of Wizardry I (1st level)
- [] Implement Ring of Wizardry II (2nd level)
- [] Implement Ring of Wizardry III (3rd level)
- [] Implement Ring of Wizardry IV (4th level)
- [] Update spell book UI to show bonus slots
- [] Test that non-arcane casters are unaffected
- [] Test that bonus stacks with ability bonuses
- [] Verify spell preparation with doubled slots

Deliverables:

- Updated CharacterStats.cs
 - Ring of Wizardry I-IV items
 - Updated spell book UI
 - Test results for all three arcane caster classes
-

PHASE 5: Regeneration (Days 17-20)

Objectives:

- Build regeneration effect component
- Implement Ring of Regeneration
- Hourly healing tracking
- Limb restoration mechanic
- Death prevention logic

Tasks:

- [] Create RegenerationEffect component
- [] Implement hourly healing timer
- [] Add HasRegenerationRing() to Character
- [] Modify CanDieFromHPLoss() logic
- [] Update TakeDamage() to handle regeneration
- [] Implement limb restoration (3d6 rounds)
- [] Add visual indicator for regeneration
- [] Test death prevention from HP loss
- [] Test massive damage still kills
- [] Test death effects still kill
- [] Create Ring of Regeneration in RingFactory

Deliverables:

- RegenerationEffect.cs

- Updated Character.cs damage logic
 - Ring of Regeneration item
 - Regeneration status UI indicator
-

PHASE 6: Polish & Integration (Days 21-22)

Objectives:

- Bug fixes
- Performance optimization
- Save/load testing
- Documentation
- Final testing

Tasks:

- [] Bug fix pass on all new systems
- [] Optimize regeneration performance (use coroutines)
- [] Test save/load with stored spells
- [] Test save/load with regeneration effect
- [] Verify all rings work together
- [] Test edge cases (multiple rings, ring swapping)
- [] Update item tooltips
- [] Write sprint 3 completion report
- [] Document new systems for future reference

Deliverables:

- Bug fix report
 - Performance metrics
 - Sprint 3 completion documentation
 - Updated technical documentation
-

TECHNICAL SPECIFICATIONS

Ring of Counterspells

Item Definition:

Name: Ring of Counterspells
Type: Ring
Price: 4,000 gp
Slot: Ring
Weight: 0 lbs

Storage Properties:

MaxStoredSpellLevels: 6
IsCounterspellRing: true
StoredSpells: List<StoredSpell>

Mechanics:

- User casts spell into ring (consumes spell slot)
- Spell must be 6th level or lower
- Ring can store only one spell at a time
- When stored spell is cast AT wearer, automatically counter
- Countering consumes the stored spell
- Functions as Dispel Magic against that spell

UI Elements:

- "Store Counterspell" button
- Dropdown to select spell
- Display currently stored spell
- "Remove Spell" button

Code Example:

```
ItemData ring = new ItemData
{
    ItemName = "Ring of Counterspells",
    ItemType = ItemType.Ring,
    Price = 4000,
    MaxStoredSpellLevels = 6,
    IsCounterspellRing = true,
    StoredSpells = new List<StoredSpell>()
};
```

Ring of Spell Storing (Minor & Major)

Minor Version:**Name:** Ring of Spell Storing, Minor**Price:** 18,000 gp**Capacity:** 3 spell levels**Examples:**

- One 3rd-level spell
- One 2nd-level + one 1st-level spell
- Three 1st-level spells

Major Version:**Name:** Ring of Spell Storing, Major**Price:** 200,000 gp**Capacity:** 5 spell levels**Examples:**

- One 5th-level spell
- One 3rd-level + one 2nd-level spell
- Five 1st-level spells

Shared Mechanics:

- Anyone can cast stored spells (even non-casters)
- Stored spell uses original caster's CL and DC
- Casting from ring does not provoke attacks of opportunity
- Storing a spell consumes a spell slot
- Stored spells remain until cast or removed

Stored Data:

- Spell name
- Spell level
- Caster level (when stored)
- Save DC (when stored)
- Unique ID (for UI management)

UI Elements:

- List of stored spells with metadata
- Capacity bar (e.g., "3/3 spell levels used")
- "Cast [Spell Name]" buttons
- "Remove [Spell Name]" buttons
- "Store New Spell" button (if capacity available)

Code Example:

```
ItemData minorRing = new ItemData
{
    ItemName = "Ring of Spell Storing, Minor",
    ItemType = ItemType.Ring,
    Price = 18000,
    MaxStoredSpellLevels = 3,
    IsCounterspellRing = false,
    StoredSpells = new List<StoredSpell>()
};

ItemData majorRing = new ItemData
{
    ItemName = "Ring of Spell Storing, Major",
    ItemType = ItemType.Ring,
    Price = 200000,
    MaxStoredSpellLevels = 5,
    IsCounterspellRing = false,
    StoredSpells = new List<StoredSpell>()
};
```



Ring of Wizardry I-IV

Type I:**Name:** Ring of Wizardry I**Price:** 20,000 gp**Effect:** Doubles all 1st-level arcane spell slots**Type II:****Name:** Ring of Wizardry II**Price:** 40,000 gp**Effect:** Doubles all 2nd-level arcane spell slots**Type III:****Name:** Ring of Wizardry III**Price:** 70,000 gp**Effect:** Doubles all 3rd-level arcane spell slots**Type IV:****Name:** Ring of Wizardry IV**Price:** 100,000 gp**Effect:** Doubles all 4th-level arcane spell slots**Requirements:**

- Only affects arcane spellcasters (Wizard, Sorcerer, Bard)
- Doubles prepared slots (Wizard) or daily slots (Sorcerer/Bard)
- Does NOT grant new spells known
- Stacks with bonus slots from high ability scores
- Multiple Wizardry rings do NOT stack with each other

Mechanics:

- Detect character class (arcane vs divine vs non-caster)
- Identify ring level (I, II, III, IV)
- Double base spell slots for that level
- Update spell preparation UI
- Update spell casting UI

Example Calculations:**Wizard (Int 16):****Base 1st-level slots:** 3**Bonus from Int:** +1**With Ring of Wizardry I:** $3 + 3 + 1 = 7$ slots**Sorcerer (Cha 18):****Base 2nd-level slots:** 4**Bonus from Cha:** +1**With Ring of Wizardry II:** $4 + 4 + 1 = 9$ slots**Cleric (Wis 14):**

Has divine spells, NOT arcane

Ring of Wizardry has NO effect

Code Example:

```

ItemData wizardryI = new ItemData
{
    ItemName = "Ring of Wizardry I",
    ItemType = ItemType.Ring,
    Price = 20000,
    Description = "Doubles all 1st-level arcane spell slots",
    SlotAffected = 1
};

// In CharacterStats.cs
public int GetSpellSlotsForLevel(int level)
{

```

```
if (!IsArcaneCaster()) return 0;

int baseSlots = CalculateBaseSlots(level);
int abilityBonus = GetBonusSlotsFromAbility(level);

// Check for Ring of Wizardry
bool hasWizardryRing = EquippedItems.Any(i =>
    i.ItemName.Contains("Ring of Wizardry") &&
    GetWizardryRingLevel(i) == level
);

if (hasWizardryRing)
{
    return (baseSlots * 2) + abilityBonus;
}

return baseSlots + abilityBonus;
}
```

Ring of Regeneration

Item Definition:**Name:** Ring of Regeneration**Type:** Ring**Price:** 90,000 gp**Slot:** Ring**Weight:** 0 lbs**CL:** 15th**Continuous Effects:****1. Healing Over Time:**

- Heal 1 HP per character level per hour
- **Examples:**
 - * **5th-level character:** 5 HP per hour
 - * **10th-level character:** 10 HP per hour
 - * **20th-level character:** 20 HP per hour

2. Limb Restoration:

- Restore severed limbs in 3d6 rounds (18-108 seconds)
- Fully functional upon restoration
- Does not restore limbs lost before equipping ring

3. Death Prevention (HP Loss Only):

- Cannot die from losing all HP
- Remains stable at negative HP (unconscious)
- Continues regenerating while unconscious
- Will eventually heal back to consciousness

Limitations:**Still Vulnerable To:**

- Massive damage (50+ damage in one hit) → Fort save DC 15
- Death effects (Finger of Death, Slay Living, etc.)
- Constitution damage reducing Con to 0
- Disintegration effects
- Petrification
- Energy drain reducing level to 0

Implementation Details:**Hourly Timer:**

- Track real-time or game-time
- Use coroutine for performance
- Pause during menus/loading

Death Check Override:

- Modify Character.TakeDamage()
- Add HasRegenerationRing() check
- Set to unconscious instead of dead
- Continue regeneration cycle

Limb Restoration:

- Track severed limbs (if system exists)
- Roll 3d6 for restoration time
- Apply regeneration coroutine
- Restore functionality

Code Example:

```
ItemData regenerationRing = new ItemData
{
    ItemName = "Ring of Regeneration",
    ItemType = ItemType.Ring,
    Price = 90000,
    Description = "Heal 1 HP per level per hour. Cannot die from HP loss.",
    CasterLevel = 15
}
```

```
};

// In Character.cs
void Start()
{
    if (HasRegenerationRing())
    {
        RegenerationEffect regen = gameObject.AddComponent<RegenerationEffect>();
        regen.Initialize(this);
    }
}

public void TakeDamage(int damage, DamageType type)
{
    CurrentHP -= damage;

    if (CurrentHP <= 0)
    {
        if (HasRegenerationRing())
        {
            SetUnconscious();
            Debug.Log($"{Name} is unconscious but regenerating...");
        }
        else
        {
            Die();
        }
    }

    // Massive damage check still applies
    if (damage >= 50)
    {
        RollFortitudeSave(15); // DC 15 massive damage save
    }
}
```

UI REQUIREMENTS

Spell Storage UI Panel

RING **OF** SPELL STORING (Minor)
Capacity: 3 spell levels

Stored Spells:

 Fireball
Caster Level: 5
Save DC: 14
Spell Level: 3
[CAST] [REMOVE]

Used: 3 / 3 spell levels
[STORE NEW SPELL] (disabled - full)

With Available Capacity:

RING **OF** SPELL STORING (Major)
Capacity: 5 spell levels

Stored Spells:

 Magic Missile
Caster Level: 3
Save DC: 11
Spell Level: 1
[CAST] [REMOVE]

Used: 1 / 5 spell levels
[STORE NEW SPELL] (4 levels available)

Features:

- Color-coded by spell level (1st=white, 3rd=yellow, 5th=red)
- Real-time capacity updates
- Disable "Store" when full
- Confirmation dialog for removing spells
- Tooltip shows spell description

Counterspell Storage UI

RING OF COUNTERSPELLS

Store One Spell (Max 6th Level)

Currently Stored:

Fireball

"Automatically counters Fireball when cast at you. One-time use."

[REMOVE COUNTERSPELL]

[CHANGE COUNTERSPELL]

Select spell to counter:

[Dropdown: Your Known Spells ▼]

[STORE COUNTERSPELL]

Empty Ring:

RING OF COUNTERSPELLS

Store One Spell (Max 6th Level)

Currently Stored: None

Select spell to counter:

[Dropdown: Your Known Spells ▼]

You must cast the spell into the ring to store it as a counterspell.

[STORE COUNTERSPELL]

Wizardry Ring Tooltip

Ring of Wizardry II
Price: 40,000 gp

Effect: Doubles all 2nd-level arcane
spell slots

Requirements: Arcane caster only
(Wizard, Sorcerer, Bard)

Current Effect:

- Base 2nd-level slots: 4
- Doubled to: 8
- Bonus from Int/Cha: +1
- Total slots: 9

✓ This ring will benefit you

For Non-Arcane Caster:

Ring of Wizardry III
Price: 70,000 gp

Effect: Doubles all 3rd-level arcane
spell slots

Requirements: Arcane caster only
(Wizard, Sorcerer, Bard)

✗ You are a Cleric (divine caster)
This ring will have NO effect on you.

Spell Book UI with Wizardry Ring

WIZARD SPELL BOOK
Eldrin the Wise

1st Level Spells
Slots: 3/7 (+3 from Ring of Wizardry I) ✨

☐ Magic Missile ☐ Shield ☐ Grease
☐ Mage Armor ☐ Identify ☐ Sleep
☐ Feather Fall

2nd Level Spells
Slots: 2/4

☐ Invisibility ☐ Scorching Ray
☐ Mirror Image ☐ Web

3rd Level Spells
Slots: 1/2

☐ Fireball ☐ Haste

Visual Enhancements:

- Cyan/blue highlight for doubled spell levels
- Sparkle effect on doubled slot count
- Tooltip explains the bonus source
- Clear indication of which ring provides bonus

Regeneration Status Indicator

Character Status Panel:

Eldrin the Wise

HP: 45 / 50 

AC: 18

Active Effects:

 Regeneration (Ring)
 +5 HP per hour
 Next heal in: 45 minutes
 Cannot die from HP loss

In Combat (Low HP):

Eldrin the Wise

HP: -3 / 50  REGENERATING

Status: Unconscious (Stable)

 Ring of Regeneration active
 Will regain consciousness when HP > 0
 Healing +5 HP per hour

 Still vulnerable to:

- Massive damage (50+)
- Death effects
- Constitution damage

Visual Effects:

- Green pulse effect on HP bar
- Heartbeat animation when regenerating
- Clear "REGENERATING" label
- Timer countdown for next heal
- Warning about vulnerabilities

TESTING PLAN

Ring of Counterspells Test Cases

- [] **Store Counterspell**
 - Cast Fireball into ring
 - Verify spell is stored
 - Verify spell slot consumed
 - Check ring tooltip shows stored spell
- [] **Automatic Counter**

- Enemy casts Fireball at wearer
 - Verify Fireball is countered
 - Check visual/audio feedback
 - Verify Fireball does no damage
 - [] **Consumption**
 - After counter, ring should be empty
 - Verify stored spell removed
 - Check ring can store new spell
 - [] **Level Limits**
 - Try to store 7th-level spell → Reject
 - Try to store 6th-level spell → Accept
 - Verify max level enforcement
 - [] **Non-Matching Spell**
 - Store Fireball
 - Enemy casts Lightning Bolt
 - Verify Lightning Bolt NOT countered
 - Verify Fireball still stored
 - [] **Save/Load**
 - Store counterspell
 - Save game
 - Load game
 - Verify counterspell still stored
-

Ring of Spell Storing Test Cases

- [] **Store Single Spell**
 - Cast Fireball (3rd level) into Minor ring
 - Verify capacity: 3/3 used
 - Check stored spell metadata (CL, DC)
- [] **Store Multiple Spells**
 - Cast Magic Missile (1st) into Major ring
 - Cast Invisibility (2nd) into same ring
 - Verify capacity: 3/5 used
 - Check both spells listed
- [] **Capacity Limits**
 - Fill Minor ring (3 levels)
 - Try to store another spell → Reject

- Check “Store New Spell” button disabled
 - [] **Cast Stored Spell**
 - Store Fireball (CL 5, DC 14)
 - Cast Fireball from ring
 - Verify uses CL 5 and DC 14
 - Verify spell removed from ring
 - [] **Non-Caster Usage**
 - Have Fighter character equip ring with stored Fireball
 - Fighter casts Fireball from ring
 - Verify spell cast successfully
 - Check uses original CL/DC
 - [] **Spell Removal**
 - Store Magic Missile
 - Remove without casting
 - Verify spell removed
 - Check capacity updated
 - [] **Minor vs Major**
 - Minor ring holds max 3 levels
 - Major ring holds max 5 levels
 - Test both capacity limits
 - [] **Save/Load**
 - Store multiple spells
 - Save game
 - Load game
 - Verify all spells preserved with metadata
-

Ring of Wizardry Test Cases

- [] **Wizard - Ring of Wizardry I**
 - Wizard (Int 14): Base 1st-level slots = 3
 - Equip Ring of Wizardry I
 - Verify slots doubled: 3 → 6
 - Check UI shows “+3 from Ring of Wizardry I”
- [] **Sorcerer - Ring of Wizardry II**
 - Sorcerer (Cha 16): Base 2nd-level slots = 4
 - Equip Ring of Wizardry II
 - Verify slots doubled: 4 → 8

- Test spell casting with doubled slots
 - [] **Stacking with Ability Bonus**
 - Wizard (Int 18): Base 1st = 3, Bonus = +1
 - Equip Ring of Wizardry I
 - Verify: $(3 \times 2) + 1 = 7$ slots
 - Check math is correct
 - [] **Non-Arcane Caster**
 - Cleric equips Ring of Wizardry I
 - Verify NO effect on spell slots
 - Check tooltip warns “No effect”
 - [] **Non-Caster**
 - Fighter equips Ring of Wizardry I
 - Verify NO effect
 - No errors or crashes
 - [] **Wrong Level Ring**
 - Wizard with Ring of Wizardry III
 - Check only 3rd-level slots doubled
 - Verify 1st and 2nd levels NOT doubled
 - [] **Multiple Wizardry Rings**
 - Equip Ring of Wizardry I and II
 - Verify both effects apply
 - 1st-level slots doubled
 - 2nd-level slots doubled
 - [] **Ring Swap**
 - Equip Ring of Wizardry I
 - Prepare 6 spells (doubled slots)
 - Unequip ring
 - Verify slots revert to 3
 - Check prepared spells handled correctly
 - [] **Save/Load**
 - Equip Ring of Wizardry I
 - Save game
 - Load game
 - Verify doubling still active
-

Ring of Regeneration Test Cases

- [] **Hourly Healing**

- Equip ring (5th-level character)
- Take 10 damage (45/50 HP)
- Wait 1 hour (game time)
- Verify healed 5 HP (50/50 HP)

- [] **Healing Amount Scales**

- Test with 1st-level character: +1 HP/hour
- Test with 10th-level character: +10 HP/hour
- Test with 20th-level character: +20 HP/hour

- [] **Death Prevention (HP Loss)**

- Take 60 damage (drop to -10 HP)
- Verify character NOT dead
- Check status: Unconscious (Stable)
- Verify regeneration continues

- [] **Recovery from Negative HP**

- At -10 HP with ring equipped
- Wait hours for regeneration
- Verify heals back to positive HP
- Check character wakes up

- [] **Massive Damage Vulnerability**

- Take 50+ damage in one hit
- Roll Fortitude save (DC 15)
- If failed, character dies (despite ring)
- Verify ring doesn't prevent this death

- [] **Death Effect Vulnerability**

- Cast Finger of Death at wearer
- Verify character can die
- Ring doesn't prevent death effects

- [] **Limb Restoration**

- Sever limb (if system exists)
- Wait 3d6 rounds
- Verify limb restored
- Check functionality restored

- [] **Ring Removal**

- Take damage to -5 HP (stable with ring)
- Remove ring

- Verify character dies immediately
 - Check death check triggers
 - [] **Save/Load**
 - Equip ring
 - Take damage
 - Save game
 - Load game
 - Verify regeneration timer preserved
-

RISK MITIGATION

Technical Risks

RISK 1: Spell Storage Complexity

- **Description:** Storing spells with full metadata (CL, DC, spell level) is complex and may have serialization issues
- **Impact:** High - Core feature of Spell Storing rings
- **Likelihood:** Medium
- **Mitigation Strategies:**
 1. Use existing spell system as foundation
 2. Create StoredSpell wrapper class for metadata
 3. Test serialization early and often
 4. Implement fallback: store spell name only, recalculate CL/DC on cast
- **Contingency Plan:** If full metadata storage fails, use simplified version where ring stores spell name and recalculates parameters based on current wearer's stats

RISK 2: Spell Slot Doubling UI Breakage

- **Description:** Modifying spell slot calculation may break existing spell preparation and casting UI
- **Impact:** High - Affects core gameplay
- **Likelihood:** High
- **Mitigation Strategies:**
 1. Create new GetSpellSlotsForLevel() method instead of modifying existing
 2. Test thoroughly with all three arcane caster classes
 3. Implement UI updates incrementally
 4. Add unit tests for slot calculation
- **Contingency Plan:** Show Wizardry ring bonus as separate line item in UI rather than integrating into base slots

RISK 3: Regeneration Performance Impact

- **Description:** Update() loop for hourly timer may impact performance, especially with multiple characters
- **Impact:** Medium - Performance degradation
- **Likelihood:** Medium
- **Mitigation Strategies:**
 1. Use coroutines instead of Update()
 2. Implement regeneration as event-based system
 3. Profile early to detect issues

4. Limit regeneration checks to active scene

- **Contingency Plan:** Move to turn-based regeneration (heal on rest) instead of real-time hourly healing

RISK 4: Counterspell Timing Issues

- **Description:** Intercepting spell casting at correct moment for counterspell may be difficult

- **Impact:** High - Core feature won't work

- **Likelihood:** Low

- **Mitigation Strategies:**

1. Hook into existing spell resolution pipeline early
2. Add counterspell check as first step in CastSpell()
3. Test with various spell types (targeted, area, instant)
4. Implement clear event system for spell interruption

- **Contingency Plan:** Change to manual activation ("Press button when enemy casts X spell") instead of automatic triggering

RISK 5: Save/Load Data Persistence

- **Description:** Complex data structures (stored spells, regeneration timers) may not save/load correctly

- **Impact:** Critical - Game breaking

- **Likelihood:** Medium

- **Mitigation Strategies:**

1. Test save/load after each system implementation
2. Use JSON serialization for complex objects
3. Implement data validation on load
4. Create save/load unit tests

- **Contingency Plan:** Implement data migration system to handle save file corruption gracefully

Scope Risks

RISK 6: Feature Creep

- **Description:** Temptation to add extra features (e.g., custom counterspell triggers, spell trading between rings)

- **Impact:** Medium - Schedule slip

- **Likelihood:** High

- **Mitigation:** Strict adherence to D&D 3.5e rules as written, defer enhancements to Sprint 4+

RISK 7: Integration Complexity

- **Description:** Four new systems may conflict with existing Sprint 1 and Sprint 2 code

- **Impact:** High - Regression bugs

- **Likelihood:** Medium

- **Mitigation:** Regression testing suite, incremental integration, code reviews

Schedule Risks

RISK 8: Underestimated Complexity

- **Description:** Spell storage or regeneration system takes longer than 4-5 days

- **Impact:** High - Sprint delay

- **Likelihood:** Medium

- **Mitigation:** Build simplest version first (MVP), add complexity incrementally, daily progress reviews
- **Contingency Plan:** Reduce scope (e.g., implement only Minor Spell Storing ring, defer Major to Sprint 4)

RISK 9: Unforeseen Bugs

- **Description:** Critical bugs discovered in Phase 6 testing
- **Impact:** High - Sprint delay
- **Likelihood:** Medium
- **Mitigation:** Test continuously during development, allocate 2 full days for bug fixing
- **Contingency Plan:** Extend sprint by 2-3 days if necessary, or defer non-critical bugs to Sprint 4

SUCCESS CRITERIA

Sprint 3 is considered **COMPLETE** when all the following criteria are met:

Implementation Criteria

- ☐ **All 6 Tier 3 rings implemented**
 - Ring of Counterspells
 - Ring of Spell Storing (Minor)
 - Ring of Spell Storing (Major)
 - Ring of Wizardry I
 - Ring of Wizardry II
 - Ring of Wizardry III
 - Ring of Wizardry IV
 - Ring of Regeneration
- ☐ **All new systems functional**
 - Spell storage system operational
 - Counterspell triggering works automatically
 - Spell slot doubling calculates correctly
 - Regeneration system heals over time

Testing Criteria

- ☐ **All test cases pass (see Testing Plan)**
 - 6 Counterspell tests
 - 8 Spell Storing tests
 - 9 Wizardry ring tests
 - 9 Regeneration tests
- ☐ **Edge cases handled**
 - Ring swapping during combat
 - Multiple rings equipped
 - Save/load with active effects
 - Non-casters using rings

Quality Criteria

- ☐ **UI displays correct information**

- Spell storage panel shows metadata
- Wizardry ring tooltip shows doubling
- Regeneration status indicator visible
- All tooltips accurate

- ☐ **Performance acceptable**

- No frame drops from regeneration
- Spell storage access < 100ms
- UI updates smooth

- ☐ **Save/load preserves data**

- Stored spells persist
- Regeneration timer preserved
- Doubled spell slots maintained
- No data corruption

Documentation Criteria

- ☐ **Code documented**

- All new classes have XML comments
- Complex logic explained
- Public methods documented

- ☐ **Technical documentation updated**

- Sprint 3 implementation notes
- Spell storage design doc
- System architecture diagrams

- ☐ **Testing results recorded**

- Test case results logged
- Bug reports filed and resolved
- Performance metrics captured

Integration Criteria

- ☐ **No regression bugs**

- Sprint 1 rings still work
- Sprint 2 rings still work
- Existing spell system unaffected

- ☐ **Code quality maintained**

- No compiler warnings
- Code review completed
- Unit tests added

TIMELINE SUMMARY

Phase	Duration	Days	Key Deliverables
Phase 1: Counter-spell	3 days	1-3	Ring of Counterspells, trigger system
Phase 2: Storage Foundation	5 days	4-8	Spell storage system, UI
Phase 3: Spell Storing Rings	3 days	9-11	Minor & Major rings, casting system
Phase 4: Wizardry Rings	5 days	12-16	4 Wizardry rings, slot doubling
Phase 5: Regeneration	4 days	17-20	Ring of Regeneration, healing system
Phase 6: Polish	2 days	21-22	Bug fixes, testing, documentation
TOTAL	22 days	(3-4 weeks)	6 rings, 4 new systems

DELIVERABLES

Code Deliverables

1. **SpellStorageSystem.cs**
 - StoredSpell class
 - Storage management methods
 - Capacity tracking
 - Serialization support
2. **CounterspellManager.cs**
 - Counterspell detection
 - Automatic triggering
 - Spell interception logic
3. **RegenerationEffect.cs**
 - Hourly healing coroutine
 - Limb restoration
 - Death prevention logic
4. **Updated CharacterStats.cs**
 - GetBonusSpellSlots()

- IsArcaneCaster()
- Modified spell slot calculation

5. **Updated SpellManager.cs**

- Counterspell checking
- Stored spell casting
- Integration hooks

6. **RingFactory.cs additions**

- Ring of Counterspells
- Ring of Spell Storing (Minor)
- Ring of Spell Storing (Major)
- Ring of Wizardry I-IV
- Ring of Regeneration

UI Deliverables

1. **SpellStoragePanel.prefab**

- Stored spell viewer
- Store/cast/remove buttons
- Capacity indicator

2. **CounterspellUI.prefab**

- Spell selection dropdown
- Storage confirmation
- Current stored spell display

3. **Updated SpellBookUI.prefab**

- Doubled spell slot display
- Visual indicators for Wizardry ring
- Tooltip enhancements

4. **RegenerationStatusIndicator.prefab**

- HP regeneration display
- Timer countdown
- Status effects panel

Documentation Deliverables

1. **sprint3_implementation_notes.md**

- Development journal
- Challenges encountered
- Solutions implemented
- Lessons learned

2. **spell_storage_design.md**

- System architecture
- Data structures
- Flow diagrams
- API reference

3. **testing_results.md**

- Test case results
- Bug reports and resolutions

- Performance metrics
- Edge case findings

4. **Updated technical_architecture.md**

- New system diagrams
- Integration points
- Data flow charts

NEXT STEPS (Post-Sprint 3)

Immediate Follow-Up

1. **Playtesting Session**

- Gather feedback on complex mechanics
- Identify usability issues
- Test balance (especially Wizardry ring costs)

2. **Performance Profiling**

- Measure regeneration system impact
- Optimize spell storage queries
- Check memory usage

3. **Documentation Review**

- Ensure all code documented
- Update wiki with new systems
- Create player-facing guide

Sprint 4 Planning

Based on Sprint 3 completion, Sprint 4 could focus on:

- **Tier 4: Metamagic Rings** (Spell Turning, Spell Resistance, etc.)
- **Tier 5: Advanced Utility** (Telekinesis, Three Wishes, etc.)
- **Polish & Balance** (UI improvements, cost adjustments)
- **Multiplayer Support** (if applicable)

NOTES

Design Philosophy

- **Faithful to D&D 3.5e:** Implement rules as written, even if counterintuitive
- **Modular Systems:** Each new system (storage, regeneration) should be reusable for future items
- **Player-Friendly UI:** Complex mechanics require clear, intuitive interfaces
- **Performance First:** Don't sacrifice framerate for features

Known Limitations

- **Real-Time Regeneration:** May need to accelerate time or use rest-based healing in some game modes
- **Spell Storage Metadata:** CL and DC preservation requires careful serialization

- **Wizardry Ring Stacking:** Rules unclear on multiple Wizardry rings; chose to allow (each affects different level)

Future Enhancements (Post-Sprint 3)

- **Spell Storage Trading:** Transfer stored spells between rings
 - **Custom Counterspells:** Store multiple counterspells with priority system
 - **Regeneration Visuals:** More elaborate healing animations
 - **UI Themes:** Customizable UI skins for different play styles
-

End of Sprint 3 Implementation Plan